



**Integrantes:**

Nicolás Alejandro Diosa Benavides

Juan Felipe Fajardo Garzón

Esteban Prieto Lugo

Juan Mateo Rozo Torres

**Grupo de trabajo:**

Grupo 4

**Docente:**

Sergio Vargas Pedraza

**Asignatura:**

Ingeniería de Software II

Universidad Nacional de Colombia

Facultad de Ingeniería

Ingeniería de Sistemas y Computación

08 de junio del 2026

## REQUERIMIENTOS FUNCIONALES

1. Como Administrador de Sistemas, quiero que el acceso se haga mediante MFA para reducir el riesgo de robo de cuentas por ataques de fuerza bruta.
2. Como Administrativo, quiero tener un registro detallado (logs) de quién modificó registros críticos para garantizar la trazabilidad.
3. Como Responsable de Cumplimiento, quiero que los datos personales de los clientes estén cifrados en la base de datos para cumplir con las normativas legales de protección de datos.

## REQUERIMIENTOS NO FUNCIONALES

1. El Sistema deberá deshabilitar por completo el Database Manager expuesto en la URL /web/database/manager para evitar que atacantes externos intenten borrar o descargar copias de seguridad.
2. El Sistema deberá ejecutar escaneos automáticos de vulnerabilidades en los módulos de terceros instalados para mitigar riesgos de código malicioso o puertas traseras (backdoors).
3. El Sistema deberá estar configurado bajo el Principio de Menor Privilegio, de modo que el usuario de la base de datos no tenga permisos de superusuario, para limitar el daño en caso de una inyección SQL.
4. El Sistema deberá realizar cifrados hacia un almacenamiento (base de datos) para garantizar la continuidad del negocio ante un ataque Man in the Middle.

## Hallazgos y conclusiones:

A partir de un análisis de código de módulos aislados del ERP Odoo, pruebas unitarias y escuchas de Wireshark se examinaron los requerimientos funcionales y no funcionales establecidos con anterioridad, pudiendo de esta manera comprender y dar un dictamen de cumplimiento de los mismos o en otro caso apuntar los fallos o vulnerabilidades que se encontraron.

### RF #1:

El ERP Odoo utiliza el método de autenticación TOTP, este método comparte una clave privada por medio de un recurso conocido como código QR al dispositivo que en cuestión quiera ser utilizado como segundo factor de autenticación, posterior a compartir la clave privada el sistema verifica un código de seis dígitos para confirmar el acceso a la cuenta o recurso del usuario. Tal clave secreta está codificada en Base64, y se utiliza un método de encriptación para generar el código de seis dígitos lo que no significa que se comunique de manera cifrada.

```

class Auth_TotpWizard(models.TransientModel):
    _name = 'auth_totp.wizard'
    _description = "2-Factor Setup Wizard"

    user_id = fields.Many2one('res.users', required=True, readonly=True)
    secret = fields.Char(required=True, readonly=True)
    url = fields.Char(store=True, readonly=True, compute='_compute_qrcode')
    qrcode = fields.Binary(
        attachment=False, store=True, readonly=True,
        compute='_compute_qrcode',
    )
    code = fields.Char(string="Verification Code", size=7, store=False)

    @api.depends('user_id.login', 'user_id.company_id.display_name', 'secret')
    def _compute_qrcode(self):
        # TODO: make "issuer" configurable through config parameter?
        global_issuer = request and request.httprequest.host.split(':', 1)[0]
        for w in self:
            issuer = global_issuer or w.user_id.company_id.display_name
            w.url = url = werkzeug.urls.url_unparse((
                'otpauth', 'totp',
                werkzeug.urls.url_quote(f'{issuer}:{w.user_id.login}', safe=':'),
                werkzeug.urls.url_encode({
                    'secret': compress(w.secret),
                    'issuer': issuer,
                    # apparently a lowercase hash name is anathema to google
                    # authenticator (error) and passlib (no token)
                    'algorithm': ALGORITHM.upper(),
                    'digits': DIGITS,
                    'period': TIMESTEP,
                }), '')
            data = io.BytesIO()
            qrcode.make(url.encode(), box_size=4).save(data, optimise=True, format='PNG')
            w.qrcode = base64.b64encode(data.getvalue()).decode()

```

Esto se puede encontrar en el módulo *auth\_totp.py* con ruta **odoo\addons\auth\_totp**, y el fragmento de código anterior en **odoo\addons\auth\_totp\wizard\auth\_totp\_wizard.py**.

Se crearon 6 tests, con el objetivo de probar funcionalidades específicas del TOTP usando el estándar 4226 y 6238:

- Test de vectores de prueba.
- Propiedades de las salidas.
- Generación de código en casos extremos.
- Pruebas de consistencia.
- Pruebas del cifrado SHA1.
- Pruebas de seguridad del secreto (clave).

**COMANDO PARA CORRER LOS TEST:** `py -3.11 odoo-bin -c .\debian\odoo.conf -d odoo_db --test-tags=auth_totp`

**Resolución:**

Odoo cumple con el principio de MFA (*Multiple Factor Authentication*), no tiene vulnerabilidades en este ámbito. Por ende cumple con el RF#1.

## RF #2:

Odoo realiza un seguimiento de los cambios que se hacen a una base de datos, esto es posible gracias al módulo `odoo\addons\mail\models\mail_thread.py` que funciona como un proceso observador o daemon, siempre que se haga cualquier tipo de operación a través del ORM (*Object Relational Mapping*) este proceso o hilo generará logs y guarda el tipo de operación, cuándo se hizo tal operación y por último qué usuario fue el que la efectuó.

```
C: > Users > Usuario > Documents > UNAL > Matricula 2026-I > Ingeniería de software II > odoo > addons > mail > models > mail_thread.py > MailThr
63 class MailThread(models.AbstractModel):
521
522     # -----
523     # TRACKING / LOG
524     # -----
525
526     def _track_prepare(self, fields_iter):
527         """ Prepare the tracking of ``fields_iter`` for ``self``.
528
529         :param iter fields_iter: iterable of fields names to potentially track
530         """
531         fnames = self._track_get_fields().intersection(fields_iter)
532         if not fnames:
533             return
534         self.env.cr.precommit.add(self._track_finalize)
535         initial_values = self.env.cr.precommit.data.setdefault(f'mail.tracking.{self._name}', {})
536         for record in self:
537             if not record.id:
538                 continue
539             values = initial_values.setdefault(record.id, {})
540             if values is not None:
541                 for fname in fnames:
542                     value = (
543                         # get the properties definition with the value
544                         # (not just the dict with the value)
545                         field.convert_to_read(record[fname], record)
546                         if (field := record._fields[fname]).type == 'properties'
547                         else record[fname]
548                     )
549                     values.setdefault(fname, value)
550
551     def _track_discard(self):
552         """ Prevent any tracking of fields on ``self``. """
553         if not self._track_get_fields():
554             return
555         self.env.cr.precommit.add(self._track_finalize)
556         initial_values = self.env.cr.precommit.data.setdefault(f'mail.tracking.{self._name}', {})
557         # disable tracking by setting initial values to None
558         for id_ in self.ids:
559             initial_values[id_] = None
560
561     def _track_filter_for_display(self, tracking_values):
562         """Filter out tracking values from being displayed."""
563         self.ensure_one()
564         return tracking_values
565
566     def _track_finalize(self):
567         """ Generate the tracking messages for the records that have been
```

## Resolución:

Odoo cumple con la generación de logs para las modificaciones y operaciones que se le hacen a una base de datos en específico por lo que sí cumple el RF#2.

### RF #3:

La seguridad referente a las bases de datos que se integren a Odoo son responsabilidad del usuario que quiera utilizar este ERP, lo que naturalmente indica que Odoo no debe preocuparse por el cifrado dentro de la base de datos, y en general puede trabajar con bases de datos cifradas y no cifradas.

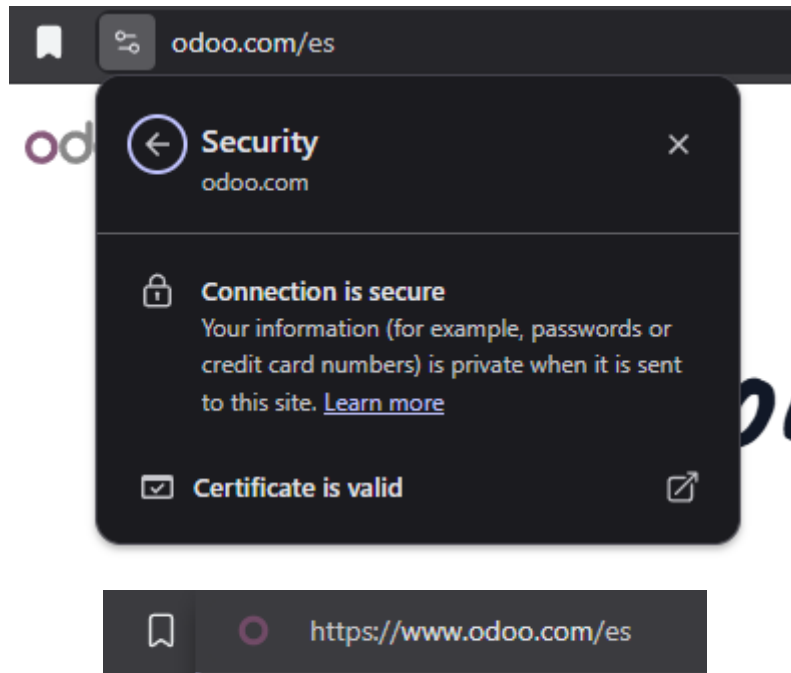
### Resolución:

Para Odoo esto no es una responsabilidad, por lo cuál este requerimiento es responsabilidad única del cliente de Odoo y de los gestores de la base de datos de la empresa en cuestión. No aplica el RF#3.

### RNF #1:

La gestión de peticiones http se hacen o reciben dentro del módulo *http.py*, en la ruta **odoo\odoo\http.py** este módulo se dedica únicamente a recibir tal información y responder a la petición comunicándose directamente con la base de datos, el nombre de la base de datos que es recibida para poder comunicarse se escribe dentro de los encabezados o headers de la petición, por lo que podría ser interceptada, pero hay que tener en cuenta que Odoo utiliza el protocolo HTTPS, que en pocas palabras cifra totalmente el contenido y agrega la capa de seguridad que haría falta para el caso de transmitir el nombre en cuestión.

```
1786 def _get_session_and_dbname(self):
1787     sid = self.httprequest._session_id_
1788     if not sid or not root.session_store.is_valid_key(sid):
1789         session = root.session_store.new()
1790     else:
1791         session = root.session_store.get(sid)
1792         session.sid = sid # in case the session was not persisted
1793
1794     for key, val in get_default_session().items():
1795         session.setdefault(key, val)
1796     if not session.context.get('lang'):
1797         session.context['lang'] = self.default_lang()
1798
1799     dbname = None
1800     host = self.httprequest.environ['HTTP_HOST']
1801     header_dbname = self.httprequest.headers.get('X-Odoo-Database')
1802     if session.db and db_filter([session.db], host=host):
1803         dbname = session.db
1804         if header_dbname and header_dbname != dbname:
1805             e = ("Cannot use both the session_id cookie and the "
1806                 "x-odoo-database header.")
1807             raise werkzeug.exceptions.Forbidden(e)
1808     elif header_dbname:
1809         session.can_save = False # stateless
1810         if db_filter([header_dbname], host=host):
1811             dbname = header_dbname
1812     else:
1813         all_dbs = db_list(force=True, host=host)
1814         if len(all_dbs) == 1:
1815             dbname = all_dbs[0] # monodb
1816
1817     if session.db != dbname:
1818         if session.db:
1819             _logger.warning("Logged into database %r, but dbfilter rejects it; logging session out.", session.db)
1820             session.logout(keep_db=False)
1821             session.db = dbname
1822
1823     session.is_dirty = False
1824     return session, dbname
```



### Resolución:

Odoo se comunica a través de HTTPS cifrando todo tipo de comunicación y petición que se mantenga, incluso en la URL no tendría que aparecer el nombre de una base de datos por lo que sí cumple con el RNF#1.

### RNF #2:

Odoo no realiza escaneos automáticos de vulnerabilidades conocidas ni hace uso de librerías como bandit, pip-audit y safety por lo tanto no hay una evaluación de riesgos asociados y por consiguiente hay susceptibilidad a una caída del sistema al no conocer la raíz de un problema que se llegue a presentar con módulos puntuales.

Sin embargo Odoo si realiza chequeos de SQL- Injection el módulo correspondiente a esta revisión se encuentra en `odoo/addons/test_lint/tests/_odoo_checker_sql_injection.py`. Que en resumen posee dos listas de consultas, una blanca (permitidas o poco riesgosas) y una negra (no permitidas), así cuando se pretenda realizar una consulta el sistema revisará que su sintaxis no pertenezca a la lista negra.

Un ejemplo de una sentencia inválida o en lista negra es la siguiente:

```
cr.execute("SELECT * FROM " + table_name)
```

La válida sería entonces:

```
cr.execute("SELECT * FROM %s", (self._table,))
```

### Resolución:

Odoo implementa parcialmente controles de vulnerabilidades conocidas, pero en general no se hacen escaneos automáticos de estas por lo que Odoo no cumple con el RNF#2.

### RNF #3:

El principio de menores privilegios (Principle of Least Privilege - PoLP) establece que cualquier componente, usuario o proceso debe tener únicamente los privilegios mínimos necesarios para realizar su función específica. En el contexto de un ERP, esto se traduce en que los usuarios deben acceder únicamente a la información y funcionalidades estrictamente necesarias para sus tareas. Esto se cumple gracias a que Odoo implementa un sistema de grupos jerárquicos con herencia de permisos, que es en síntesis una manera de que usuarios de mayor rango tengan los permisos también de los usuarios de menor rango.

Dentro de los archivos y rutas siguientes se encuentran tanto la definición de privilegios como de los grupos que gozan de ellos:

- **odoo/odoo/addons/base/models/res\_groups.py**
- **odoo/odoo/addons/base/models/res\_groups\_privilege.py**
- **odoo/odoo/addons/base/models/res\_users.py**

Cada grupo en el sistema posee listas de control de acceso (*rwcd*) que define sus permisos que tiene sobre un módulo o tabla/registro del ERP.

```
class ResGroups(models.Model):
    view_group_hierarchy = fields.Json(string='Technical field for default group setting', compute='_compute_view_group_hierarchy')

    _name_uniq = models.Constraint("UNIQUE (privilege_id, name)",
        'The name of the group must be unique within a group privilege!')
    _check_api_key_duration = models.Constraint(
        'CHECK(api_key_duration >= 0)',
        'The api key duration cannot be a negative value.',
    )

    """ The groups involved are to be interpreted as sets.
    Thus we can define groups that we will call for example N, Z... such as mathematical sets.

    C
    R
    Q
    Z
    N
    P
    I

    "C"
    "I" implied "C"
    "R" implied "C"
    "Q" implied "R"
    "P" implied "R"
    "Z" implied "Q"
    "N" implied "Z"

    For example:
    * A manager group will imply a user group: all managers are users (like Z imply C);
    * A group "computer developer employee" will imply that he is an employee group, a user
    group, that he has access to the timesheet user group.... "computer developer employee"
    is therefore a set of users in the intersection of these groups. These users will
    therefore have all the rights of these groups in addition to their own access rights.

    implied_ids = fields.Many2many('res.groups', 'res_groups_implied_rel', 'gid', 'hid',
        string='Implied Groups', help='Users of this group are also implicitly part of those groups')
    all_implied_ids = fields.Many2many('res.groups', string='Transitively Implied Groups', recursive=True,
```

Cabe aclarar que existen reglas globales para elementos o usuarios que pertenecen y no pertenecen a grupos puntuales, por defecto se aplica denegación (si no posee una Access Control List especificada no puede acceder a ningún recurso).

## Resolución:

El sistema Odoo cumple con jerarquizar privilegios según roles específicos en el sistema para no exceder las capacidades necesarias de un usuario específico por lo que cumple con el RNF#3.

## RNF #4:

Se realizó una captura de paquetes de datos a través de Wireshark en el momento de la ejecución de una prueba unitaria de conexión a una base de datos local en PostgreSQL, basada en el módulo *auth\_totp.py* con el objetivo de examinar si tal información intercambiada entre la base de datos y el módulo estaría cifrada o no, el resultado fue el siguiente:

| No. | Time      | Source | Destination | Protocol | Length | Info                                 |
|-----|-----------|--------|-------------|----------|--------|--------------------------------------|
| 462 | 38.255638 | ::1    | ::1         | PGSQL    | 72     | >?                                   |
| 464 | 38.276995 | ::1    | ::1         | PGSQL    | 65     | <                                    |
| 466 | 38.277080 | ::1    | ::1         | PGSQL    | 129    | >?                                   |
| 468 | 38.297327 | ::1    | ::1         | PGSQL    | 522    | <R/S/S/S/S/S/S/S/S/S/S/S/S/S/S/S/K/Z |
| 470 | 38.297654 | ::1    | ::1         | PGSQL    | 118    | >Q                                   |
| 472 | 38.297790 | ::1    | ::1         | PGSQL    | 81     | <C/Z                                 |
| 474 | 38.297816 | ::1    | ::1         | PGSQL    | 127    | >Q                                   |
| 476 | 38.300145 | ::1    | ::1         | PGSQL    | 135    | <T/D/C/Z                             |
| 478 | 38.300348 | ::1    | ::1         | PGSQL    | 78     | >Q                                   |
| 480 | 38.300446 | ::1    | ::1         | PGSQL    | 84     | <C/Z                                 |
| 482 | 38.300536 | ::1    | ::1         | PGSQL    | 69     | >X                                   |
| 490 | 39.006050 | ::1    | ::1         | PGSQL    | 72     | >?                                   |
| 492 | 39.016729 | ::1    | ::1         | PGSQL    | 65     | <                                    |
| 494 | 39.016819 | ::1    | ::1         | PGSQL    | 128    | >?                                   |
| 496 | 39.019983 | ::1    | ::1         | PGSQL    | 522    | <R/S/S/S/S/S/S/S/S/S/S/S/S/S/S/K/Z   |
| 498 | 39.020252 | ::1    | ::1         | PGSQL    | 118    | >Q                                   |
| 500 | 39.020478 | ::1    | ::1         | PGSQL    | 81     | <C/Z                                 |
| 502 | 39.020529 | ::1    | ::1         | PGSQL    | 261    | >Q                                   |
| 504 | 39.025182 | ::1    | ::1         | PGSQL    | 121    | <T/C/Z                               |
| 506 | 39.025301 | ::1    | ::1         | PGSQL    | 129    | >Q                                   |
| 508 | 39.025599 | ::1    | ::1         | PGSQL    | 143    | <T/D/C/Z                             |
| 510 | 39.025653 | ::1    | ::1         | PGSQL    | 78     | >Q                                   |
| 512 | 39.025698 | ::1    | ::1         | PGSQL    | 84     | <C/Z                                 |
| 514 | 39.025829 | ::1    | ::1         | PGSQL    | 81     | >Q                                   |
| 516 | 39.025879 | ::1    | ::1         | PGSQL    | 87     | <C/Z                                 |
| 518 | 39.025968 | ::1    | ::1         | PGSQL    | 118    | >Q                                   |
| 520 | 39.025998 | ::1    | ::1         | PGSQL    | 81     | <C/Z                                 |
| 522 | 39.026016 | ::1    | ::1         | PGSQL    | 348    | >Q                                   |
| 524 | 39.030917 | ::1    | ::1         | PGSQL    | 344    | <T/D/D/D/D/D/D/D/C/Z                 |

Frame 462: Packet, 72 bytes on wire (576 bits), 72 bytes captured (576 bits) on interface \Device\NPF\_{Loopback}, id 0

Null/Loopback

Internet Protocol Version 6, Src: ::1, Dst: ::1

Transmission Control Protocol, Src Port: 58152, Dst Port: 5432, Seq: 1, Ack: 1, Len: 8

PostgreSQL



```
Wireshark - Seguir secuencia TCP (tcp.stream eq 36) - Adapter for loopback traffic capture

...@...user.odoo.database.odoo_db.application_name.odoo-16412...
R.....S...in_hot_standby.off.S...integer_datetimes.on.S...TimeZone.America/Bogota.S...IntervalStyle.postgres.S...search_path."$user", public.S...is_superuser.off.S...
application_name.odoo-16412.S...&default_transaction_read_only.off.S...scram_iterations.4096.S...DateStyle.ISO, MDY.S...#standard_conforming_strings.on.S...session_authori
tion.odoo.S...client_encoding.UTF8.S...server_version.18.4.S...server_encoding.UTF8.K.....$Z...
Q...SBEGIN ISOLATION LEVEL REPEATABLE READ READ WRITE.
C...
BEGIN.Z...T
Q...
SELECT p.provolatile
FROM pg_proc p
WHERE p.proname = 'unaccent'
AND p.pronamespace = current_schema::regnamespace
AND p.pronargs = 1
T...$.provolatile.....C...
SELECT 0.Z...T
Q...@SELECT proname FROM pg_proc WHERE proname='word_similarity'.
T...proname.....@.....D.....word_similarityC...
SELECT 1.Z...T
Q...
ROLLBACK.
C...
ROLLBACK.Z...T
Q...DISCARD ALL.
C...DISCARD ALL.Z...T
Q...SBEGIN ISOLATION LEVEL REPEATABLE READ READ WRITE.
C...
BEGIN.Z...T
Q...SELECT table_name FROM information_schema.tables WHERE table_name IN ('orm_signaling_registry', 'orm_signaling_default', 'orm_signaling_assets', 'orm_signaling_stable', 'o
rm_signaling_templates', 'orm_signaling_routing', 'orm_signaling_groups') AND table_schema = current_schema.
T...#.table_name.....@.....D.....orm_signaling_registryD.....orm_signaling_defaultD.....orm_signaling_assetsD.....orm_signaling_stableD.....orm
signaling_templatesD.....orm_signaling_routingD.....orm_signaling_groupsC...
SELECT 7.Z...T
Q...ZSELECT (SELECT max(id) FROM "orm_signaling_registry"), (SELECT max(id) FROM "orm_signaling_default"), (SELECT max(id) FROM "orm_signaling_assets"), (SELECT max(id) FROM
"orm_signaling_stable"), (SELECT max(id) FROM "orm_signaling_templates"), (SELECT max(id) FROM "orm_signaling_routing"), (SELECT max(id) FROM "orm_signaling_groups").
T.....max.....max.....max.....max.....max.....max.....D.....8...
1.....1.....1.....1.....1C...
SELECT 1.Z...T
Q...COMMIT.
C...COMMIT.Z...T
Q...DISCARD ALL.
C...DISCARD ALL.Z...T
Q...SBEGIN ISOLATION LEVEL REPEATABLE READ READ WRITE.
C...
BEGIN.Z...T
Q...
SELECT c.relname
FROM pg_class c
WHERE c.relname IN ('ir_module_module')
AND c.relkind IN ('r', 'v', 'm')
AND c.relnamespace = current_schema::regnamespace
T...relname.....@.....D.....ir_module_moduleC...
SELECT 1.Z...T
Q...DELETE FROM ir_config_parameter WHERE key='base.partially_updated_database'.
C...
DELETE 0.Z...T
Q...COMMIT.
C...COMMIT.Z...T
```

Lo que es un hallazgo de una vulnerabilidad clara, pues las consultas a la base de datos no están cifradas por lo que ante un ataque de Man in the Middle existiría una probabilidad alta de robo de credenciales e información sensible, se eligió este módulo (TOTP) pues es donde se materializa la autenticación de dos factores por lo que podrían ser vulneradas las barreras de autenticación a partir únicamente de capturar el intercambio de información con la base de datos.

## Resolución:

Odoo no cifra la comunicación interna entre la base de datos y el módulo de autenticación den dos factores TOTP, por lo que existe una vulnerabilidad de alto riesgo y no cumple con el RNF#4.